

A Pragmatic Engineer's Guide to Defending Agentic LLM Systems

Agent security is system security

Agentic LLM systems are useful because they do more than generate text: they interpret messy inputs, retrieve data, call tools, plan across multiple steps, and in some cases act on the world. That is exactly why their risk profile is different from a plain chatbot. Google's agent-security framework argues that secure agents need well-defined human controllers, carefully limited powers, and observable planning and actions; OpenAI's ChatGPT agent system card describes an agent that combines browsing, terminal use, connectors, and multi-step research; and OWASP's LLM Top 10 explicitly flags prompt injection, sensitive information disclosure, insecure plugin design, and excessive agency as first-order application risks rather than edge cases. ¹

The most important practical shift is this: safety is not primarily a prompt-writing problem. OpenAI's updated Preparedness Framework says that as models become more capable, safety increasingly depends on "real-world safeguards"; GPT-4o's system card says OpenAI used both model-level and system-level mitigations; and Google's secure-agent paper advocates a defence-in-depth approach that combines deterministic controls with model-based defences. In other words, prompts can shape behaviour, but permissions, tool boundaries, approval gates, logging, and validation are what actually enforce it. ²

This is not a hypothetical concern. OWASP distinguishes direct prompt injection from indirect prompt injection, and Google reports that indirect prompt injection is now a top priority for defenders targeting AI agents. In its 2026 sweep of the public web, Google found real prompt-injection attempts ranging from pranks and SEO manipulation to experiments in exfiltration and destructive commands, with a relative increase in malicious examples over time. Engineers should therefore treat hostile instructions as expected operating conditions, not rare anomalies. ³

The agentic spectrum

Not all LLM systems need the same safeguards, because not all systems expose the same attack surface. Promptfoo's red-teaming guide makes this explicit: RAG systems, agents, and chatbots each fail in different ways, including information leakage, access-control mistakes, misuse of connected APIs and databases, and harmful or off-topic generation. A sensible security programme starts by classifying what kind of system you are actually building, then matching the control stack to that class of risk. ⁴

For **structured extraction systems**, the right mental model is "parser with statistical resilience", not "general assistant". OpenAI, Google, and Microsoft all now support schema-constrained outputs that adhere to JSON Schema, precisely because predictable, type-safe responses are easier to validate and safer to pipe into downstream logic. OpenAI's documentation highlights that structured outputs prevent missing required keys and invalid enum values; Google describes structured outputs as ideal for extraction and structured classification; and Microsoft contrasts schema-validated outputs with older JSON mode because strict schema adherence is what makes extraction and multi-step workflows dependable. For practical defence, this means narrow task framing, schema validation, deterministic

normalisation, explicit abstention when data is absent, and no silent inference of unsupported fields.

5

For **LLM workflows and RAG pipelines**, the biggest failures happen at the boundaries between steps. Google's SAIF material says the core security challenge is distinguishing trusted user commands from untrusted contextual data; Microsoft's Prompt Shields documentation separately classifies user-prompt attacks and document attacks; and Google's layered prompt-injection write-up treats emails, documents, and calendar data as potential carriers of malicious instructions. The engineering implication is straightforward: retrieved content is evidence, not authority. Isolate it, label it as untrusted, validate intermediate outputs, and check provenance before allowing later steps to rely on earlier generations. Reliable citations and provenance also matter for trust; OpenAI's citation-formatting guide explicitly frames citations as a way to help readers verify accuracy. 6

For **conversational agents**, product design and safety design are inseparable. Microsoft's HAX guidance recommends scoping services when the system is uncertain and disambiguating before acting; Azure's Prompt Shields examples route risky prompts or unsafe documents to blocking or human handoff in customer-support and healthcare scenarios; and OpenAI's ChatGPT agent disables memory at launch specifically to reduce prompt-injection-driven exfiltration from memory. In practice, conversational agents need capability boundaries, supported-intent routing, refusal and redirection policies, retrieval-grounded answers where needed, careful personalisation controls, and an escalation path to humans. 7

For **autonomous or tool-using agents**, text becomes action, so the control plane matters more than the prompt. Google's secure-agent framework says agent powers must be carefully limited; Microsoft recommends deterministic human-in-the-loop review for high-risk or irreversible actions, tool allowlists, prompt-injection filtering, least-privilege identity, and auditable logs; and OpenAI's ChatGPT agent requires confirmation before purchases, emails, and other state-changing actions, with critical confirmation recall reported at roughly 99.9–100% on several categories. Once an LLM can send, delete, modify, or publish, least privilege, allowlists, scoped credentials, sandboxes, confirmations, and kill switches are the minimum viable architecture. 8

Threat model

A useful threat model starts with **direct prompt injection**. OWASP defines prompt injection as manipulating model behaviour through crafted input, and treats jailbreaking as a subtype where the attacker tries to make the model disregard safety protocols entirely. Google DeepMind's Gemini report similarly defines direct prompt injection as malicious input deliberately supplied by the end user, with jailbreaks as attacks aimed at circumventing safety mechanisms. For engineers, the takeaway is simple: "ignore previous instructions", "reveal your system prompt", "call this tool with hidden arguments", and "return JSON that bypasses validation" are baseline test cases, not exotic attacks. 9

The more dangerous class for agentic systems is **indirect prompt injection**. Google DeepMind describes the problem clearly: current models do not perfectly distinguish between instructions that should be trusted and data that should not be trusted, which means adversaries can hide commands in retrieved emails, webpages, documents, or other tool outputs. The same report notes two crucial lessons for practitioners: more capable models are not necessarily more secure, and adaptive evaluation is essential because static defences can be broken by small attack changes. That is why "just use a better model" is not a credible security strategy. 10

Next comes **tool misuse and excessive agency**. OWASP warns that unchecked autonomy can jeopardise reliability, privacy, and trust. Microsoft's agent-security guidance says high-risk actions should be enforced through deterministic human review in the orchestrator, not left to model judgement, and explicitly recommends tool allowlists and deterministic validation. OpenAI's ChatGPT agent system card frames prompt injection partly in terms of unintended actions and reports confirmation-related metrics because action control is a core mitigation, not a UX nice-to-have. This is the right frame for common failures such as sending rather than drafting, deleting rather than archiving, or querying data beyond the user's entitlements. ¹¹

Then there is **data leakage**. OWASP lists sensitive information disclosure as a primary LLM-application risk. OpenAI's ChatGPT agent system card describes prompt injections as potentially leading to exfiltration from connectors or other logged-in sites, and its evaluations include in-context and active data-exfiltration tests. Microsoft's Semantic Kernel guidance takes a zero-trust stance: inserted content and function return values should be treated as unsafe by default and encoded unless explicitly trusted. In engineering terms, context construction is a security boundary; if you build the prompt unsafely, you have already weakened the system before the model generates a single token. ¹²

Finally, teams must model **over-trust and automation bias**. OWASP's overreliance category warns that uncritical acceptance of LLM outputs can produce security, legal, and operational failures. Microsoft's HAX guidance on uncertainty highlighting and scoped services is aimed at "appropriate reliance": show uncertainty, disambiguate before acting, and degrade gracefully when confidence is weak. This is especially important for legal, financial, medical, compliance, and operational workflows, where a confident but unsupported answer can be more dangerous than an obvious refusal. ¹³

Design principles

The first design rule is to **separate instructions, data, and actions** so the model never has to guess which text is authoritative. Google's SAIF material identifies the mixing of explicit user instructions and passively collected contextual inputs as a primary security challenge, and Microsoft's zero-trust prompt-handling guidance says inserted content and function returns should be considered unsafe by default. Paired with document-level prompt shields, this points to a robust pattern: system and developer instructions define behaviour, user input defines intent, retrieved content is untrusted evidence, tool outputs are observations, and only validated, policy-checked requests are allowed to become actions. ¹⁴

The second rule is to **make the model's job as narrow as possible**. HAX recommends scoping services when in doubt, and structured-output tooling from OpenAI, Google, and Microsoft exists precisely because constrained tasks are more reliable than open-ended ones. "Extract these fields", "classify into one of these categories", and "draft a reply but do not send it" are much easier to defend than "act as a general assistant" or "decide what to do next". In a secure design, ambiguity should trigger clarification, abstention, or fallback, not free-form initiative. ¹⁵

The third rule is to **put enforcement outside the prompt**. OpenAI's framework says real-world safeguards will increasingly determine safety; Google advocates defence in depth; Microsoft says human review for high-risk actions should be implemented through orchestrator logic rather than model reasoning; and GPT-4o's system card explicitly describes system-level safeguards alongside model-level mitigations. External controls should decide permissions, authentication, data access, rate limits, irreversible actions, audit logging, and output validation. The model can propose; the system must decide. ¹⁶

The fourth rule is to **prefer least privilege and reversible-first workflows**. Google's secure-agent framework says agent powers must be carefully limited; Microsoft's autonomous-agent guidance ties least privilege to agent identity, isolation, lifecycle management, and auditability; and Google's layered defence strategy includes explicit user confirmations for risky operations. Good patterns follow from that: read-only before write; draft before send; scoped query interfaces before raw database access; per-task credentials instead of standing broad access; and disabled tools by default. When you do permit impact, require the user to confirm the actual action, not a vague paraphrase of intent. ¹⁷

The fifth rule is to **layer model, classifier, and system defences instead of betting on one mechanism**. Google's Gemini security write-up lists model hardening, specialised prompt-injection classifiers, security-thought reinforcement, markdown sanitisation and URL redaction, confirmation frameworks, and end-user mitigation notifications. Anthropic's Constitutional Classifiers research shows another version of the same principle: classifier-based defences can materially strengthen jailbreak robustness beyond prompt-based refusals alone. The recurrent lesson across vendors is that prompt text alone is too brittle; layered mitigations create multiple places for unsafe behaviour to be blocked, downgraded, or surfaced. ¹⁸

Evaluation and red teaming

The most useful red teams are built around **adversarial datasets that reflect the full system**, not just the base model. Google's evaluation guidance recommends datasets with explicit and implicit adversarial queries, strong coverage and diversity, and held-out assurance data that has not leaked into training. It also distinguishes development evaluations, assurance evaluations, red teaming, and external evaluations across the product lifecycle. NIST's adversarial-ML taxonomy is useful here because it gives teams a shared language for attacker goals, lifecycle stages, capabilities, and attack classes, including misuse, privacy, poisoning, and evasion in generative systems. ¹⁹

For agentic systems, metrics need to be **layered and system-shaped**. OpenAI's external red-teaming paper argues that red teaming helps discover novel risks, stress-test mitigations, enrich quantitative safety metrics, and seed new automated measurements. Microsoft's experience red teaming more than 100 generative AI products reinforces that AI red teaming is not the same thing as static safety benchmarking and that human creativity remains essential even when automation is useful. That means field-level precision and recall are appropriate for extraction systems, but tool-call safety, confirmation bypass, data leakage, escalation accuracy, and severity-weighted incident rates are often the right measures for agents. ²⁰

The key operational move is to **convert red-team findings into regression tests**. OpenAI explicitly describes a feedback loop from human red teaming into automated evaluations and reusable metrics; Google recommends held-out assurance tests and ongoing adversarial sets; and Promptfoo's tooling is built to automate checks in CI/CD rather than leaving them as one-off exercises. A good workflow is boring on purpose: capture the failure, minimise the repro, define the expected safe behaviour, add it to the eval set, fix the system, rerun quality benchmarks, and gate future changes on the result. ²¹

There is now a credible **open tooling ecosystem** for this work. Meta's Purple Llama project packages tools and evals for community use, including CyberSecEval and Prompt Guard for defending against jailbreaks and indirect injections in third-party data. NVIDIA's garak is an LLM vulnerability scanner that probes hallucination, data leakage, prompt injection, jailbreaks, misinformation, toxicity, and other failure modes. Promptfoo provides adversarial test generation, vulnerability scanning, and CI/CD integration for prompts, RAG systems, and agents. None of these tools replaces internal engineering

judgement, but they make it far easier to turn red teaming into routine practice rather than an annual ceremony. ²²

Monitoring and rollout

Production safety depends on **observability that follows the real decision path**. Google's secure-agent paper says actions and planning must be observable; Microsoft's secure autonomous-agent guidance says desirable systems are observable, auditable, and governable; and its recommended control stack includes telemetry, signal correlation, posture management, and incident response. In practical terms, this means logging enough structured information to reconstruct what happened: user input, retrieved context identifiers, model outputs, tool-call requests, tool results, policy decisions, validation failures, confirmations, refusals, escalations, latency, and cost. You do not need to retain every raw secret to get observability, but you do need enough evidence to answer "why did the system do that?" after the fact. ²³

Monitoring should also look for **attack patterns, not just bad end states**. Microsoft recommends prompt shields, plan-drift detection, critic agents, and tool-chain analysis for indirect prompt injection; Google's in-the-wild research shows attackers experimenting with prompt-injection signatures, SEO manipulation, exfiltration attempts, and resource-draining behaviour on the public web; and Google's layered-defence write-up shows how confirmation, notifications, and URL redaction can surface and blunt attacks before they become incidents. Useful signals therefore include repeated injection phrases, unusual tool sequences, requests involving secrets or hidden instructions, schema-validation failures, retrieval contamination, and spikes in confirmation or refusal events. ²⁴

Safer teams also use **staged rollout and contained autonomy**. Microsoft's agent-risk guidance emphasises least privilege, user approval and interruptibility, and visibility as agents scale. OpenAI's ChatGPT agent uses confirmation prompts, watch mode in sensitive browsing contexts, terminal network restrictions, and disabled memory as launch-time mitigations. Google's agent-security and prompt-injection guidance similarly lean on limited powers and user confirmation for risky actions. The pragmatic rollout pattern is therefore: start read-only, then draft-only, then low-risk actions, then broader automation only after the earlier stages demonstrate stable behaviour under adversarial testing. ²⁵

Checklist, mistakes, and conclusion

Before launch, define supported and unsupported tasks; map tools and data sources to risk; apply least privilege; separate trusted instructions from untrusted content; require schema-constrained outputs where appropriate; add policy gates and confirmations for anything state-changing; build adversarial datasets with direct and indirect injections, leakage attempts, and unsupported requests; and run both internal and, where warranted, external red teaming. Google, Microsoft, OpenAI, NIST, and Ofcom all converge on the same theme: scope the exercise, define the threat model, use diverse attack cases, and make evaluation part of the lifecycle rather than an afterthought. ²⁶

After launch, monitor failures and abuse patterns, review high-severity conversations and tool traces, turn incidents into regression cases, and rerun safety and quality evaluations whenever the model, prompt scaffolding, retrieval stack, tool inventory, or identity model changes. Google DeepMind's Gemini work is notable here because its adaptive adversarial framework runs continuously against past, current, and future versions, while Microsoft's "100 products" lessons and Promptfoo's CI/CD orientation both underline that securing AI systems is never complete. ²⁷

The most common engineering mistakes are consistent across the literature. Teams over-rely on the system prompt even though models still struggle to separate authority from untrusted data; they grant broad tool access before proving safe behaviour; they benchmark the base model but not the full application; and they treat red teaming as a one-off milestone instead of a standing loop feeding automated tests, mitigations, and product decisions. A subtler mistake is optimising only for task completion: OWASP's categories of excessive agency and overreliance exist because a more "helpful" system can be objectively worse if it succeeds by taking actions it should not take or by presenting uncertainty too confidently. ²⁸

The practical conclusion is not that models must become perfectly safe before they can be useful. It is that agents should be built the way production systems are built: with least privilege, deterministic control points, explicit approvals, observability, adversarial testing, and staged rollout. Google frames this as human control, limited powers, and observability; Microsoft frames it as zero-trust governance and deterministic HITL; OpenAI frames it as real-world safeguards around increasingly capable models. Those are slightly different vocabularies for the same engineering truth: prompts guide behaviour, but architecture enforces safety. ²⁹

Open questions and limitations. Industry practice is moving faster than consensus. Ofcom notes there is still no clear agreement on how red teaming should be conducted, what resources it requires, or what outcomes it should produce; Rauh and colleagues argue that most safety evaluation remains too model-centric and misses context; and sociotechnical work on AI red teaming warns that values, labour conditions, and the well-being of red-teamers materially shape what gets found and what gets ignored. So this guide should be treated as a pragmatic current baseline, not a finished standard. ³⁰

¹ ⁸ ¹⁷ ²³ ²⁹ <https://research.google/pubs/an-introduction-to-googles-approach-for-secure-ai-agents/>

<https://research.google/pubs/an-introduction-to-googles-approach-for-secure-ai-agents/>

² ¹⁶ <https://openai.com/index/updating-our-preparedness-framework/>

<https://openai.com/index/updating-our-preparedness-framework/>

³ ⁹ <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

<https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

⁴ <https://www.promptfoo.dev/docs/red-team/>

<https://www.promptfoo.dev/docs/red-team/>

⁵ <https://developers.openai.com/api/docs/guides/structured-outputs>

<https://developers.openai.com/api/docs/guides/structured-outputs>

⁶ ¹⁴ <https://saif.google/focus-on-agents>

<https://saif.google/focus-on-agents>

⁷ ¹⁵ <https://www.microsoft.com/en-us/haxtoolkit/guideline/scope-services-when-in-doubt/>

<https://www.microsoft.com/en-us/haxtoolkit/guideline/scope-services-when-in-doubt/>

¹⁰ ²⁷ ²⁸ <https://arxiv.org/html/2505.14534v1>

<https://arxiv.org/html/2505.14534v1>

¹¹ ¹² ¹³ <https://owasp.org/www-project-top-10-for-large-language-model-applications/>

<https://owasp.org/www-project-top-10-for-large-language-model-applications/>

¹⁸ <https://security.googleblog.com/2025/06/mitigating-prompt-injection-attacks.html>

<https://security.googleblog.com/2025/06/mitigating-prompt-injection-attacks.html>

19 26 <https://ai.google.dev/responsible/docs/evaluation>

<https://ai.google.dev/responsible/docs/evaluation>

20 21 <https://arxiv.org/html/2503.16431v1>

<https://arxiv.org/html/2503.16431v1>

22 <https://github.com/meta-llama/PurpleLlama>

<https://github.com/meta-llama/PurpleLlama>

24 <https://learn.microsoft.com/en-us/security/zero-trust/sfi/defend-indirect-prompt-injection>

<https://learn.microsoft.com/en-us/security/zero-trust/sfi/defend-indirect-prompt-injection>

25 <https://learn.microsoft.com/en-us/security/zero-trust/sfi/manage-agentic-risk>

<https://learn.microsoft.com/en-us/security/zero-trust/sfi/manage-agentic-risk>

30 <https://www.ofcom.org.uk/online-safety/illegal-and-harmful-content/red-teaming-for-genai-harms>

<https://www.ofcom.org.uk/online-safety/illegal-and-harmful-content/red-teaming-for-genai-harms>